Crabs on camera: project documentation

Gina Brøten

2026-08-13

Contents

Chapter 1

Background

1.1 Project and repository aims

The basis of this project is rooted in the work done in the masters thesis by Brøten [2026]. The aim of the project was to improve monitoring of the data-limited brown crab (*Cancer pagurus*) stock in Norway using machine learning and underwater video surveys.

The aim of this repository is not to push the boundaries of machine learning or showcase new methods, rather to make these methods more approachable for marine scientists working in fisheries applications.

1.2 About the repository

This repository uses the same code, functions, and packages developed for the masters thesis, though some functions have been simplified here to make the pipeline easier to reproduce. The structure of the repository is as follows:

```
crabs-on-camera/
  data/
      annotations/
          test_subset/              # example dataset shipped with the repo
              annotations/
                  instances_default.json   # CVAT/COCO export: labels + crab outlines
              images/default/               # the video frames those annotations belong to
  script/
      raw_to_yolo.ipynb             # converts raw annotations into a YOLO-ready dataset
      build_test_subset.ipynb       # builds a smaller/more diverse test set from a larger one
      annotation_stats.ipynb        # summary statistics on the annotation data
      train.ipynb                   # trains the YOLO model
```

```
    annotation_format              # template CVAT label schema, for annotating your
docs/                              # this documentation
README.md
```

Everything needed to reproduce the pipeline lives in three places: `data/` holds a small data set you can run the scripts against immediately, `script/`holds the actual pipeline as a series of notebooks, and `docs/` is the code documentation.

To try it yourself, clone this repository, either on your own computer for the small example dataset, or on a server with GPU access if you plan to train on a larger dataset.

AI tools (large language models) were used throughout this repository to help generate code, fix syntax, and clean up errors.

1.3 Glossary

- **Object detection**: a computer vision task where a model finds where objects are in an image and identifies what they are — in this project, locating and identifying crabs in video frames.
- **Bounding box**: a rectangular region defined by four coordinates that encloses a specific object within an image or frame.
- **RLE mask** (Run-Length Encoded mask): a pixel-by-pixel outline of an object's exact shape, stored in a compressed format — this is how the original CVAT annotations are drawn, before being converted to bounding boxes.
- **Annotation**: a label marking an object within an image — in CVAT, drawn as a mask or box around each crab.
- **Label**: the concrete tag applied to an annotation — in this context, species (e.g. cod).
- **Class**: the abstract category an object belongs to — in this context, species group/family (e.g. gadoid).
- **IoU** (Intersection over Union): a measure of how much two boxes overlap, used both to remove duplicate boxes during annotation cleanup and later as an evaluation metric.
- **Fold**: one of the K train/validation splits created during K-fold cross-validation.
- **K-fold cross-validation**: splitting the dataset into K equal-sized groups ("folds"). In each round, one fold is held out for validation while the model trains on the rest, repeated K times so every image gets used for both training and validation, just never in the same round. Example: with 5-fold cross-validation, the model is trained and validated 5 separate times, each on a different split, so results don't depend on one lucky (or unlucky) train/validation split.
- **Stratified sampling**: splitting data so each subset keeps roughly the same proportion of a key category as the full dataset. Example: if 30% of

all images contain a crab, stratified sampling makes sure each fold's train and validation sets are also close to 30% crab-present, instead of one fold accidentally ending up almost entirely empty frames.

- **Station leakage**: when images from the same BRUV station appear in both the training and validation sets, letting the model partly be tested on footage from a site it already trained on, this project's K-fold export explicitly prevents it by grouping all images from a station together.
- **Negative example / empty frame**: a frame with no crab in it, deliberately kept in the dataset (rather than discarded) because training on empty frames alongside crab-present ones has been shown to improve performance.
- **Ground truth**: the human-annotated "correct answer" (the CVAT masks/boxes) that a model's predictions are compared against.
- **YAML**: a plain-text file format for configuration settings, structured by indentation rather than punctuation: used here to tell the training script where the data lives and what classes to detect.

1.4 Before running the scripts

1.4.1 Prerequisites

All packages required for data transformation and object detection training are open-source and freely available.

To identify and quantify crabs in the videos, this project uses the object detection framework "You Only Look Once" (YOLO), specifically version 8 [Jocher et al., 2023], which builds on the original YOLO architecture introduced by Redmon et al. [2016]. Several object detection frameworks, including newer YOLO versions, could have been used instead. YOLOv8 was chosen because it is well documented and its common issues are well known, which makes it easier to troubleshoot for beginners in machine learning.

Python must be installed. `raw_to_yolo.ipynb` checks for missing packages and installs them automatically (see Chunk 0).

A server with GPU access is recommended if you plan to train on a large dataset — the small example dataset shipped with this repo can be run locally without one.

If you want to train a detector on your own data, you'll need an annotated dataset structured the same way as the example dataset: an `annotations` folder containing the CVAT/COCO export (`instances_default.json`), and an `images/default` folder containing the matching image files.

1.4.2 Data

The repo only ships a small example/test dataset (`data/annotations/test_subset`) so the scripts can be run and verified without downloading everything.

The dataset used here for testing is from the Institute of Marine Research inshore gill- and fyke-net survey conducted in 2025. The material is from ten Baited Remote Underwater Video (BRUV) stations. For more stats about the annotation composition, run `annotation_stats`.

Annotations are exported from CVAT, as a .JSON file, which is a plain text file storing structured data, in this case annotations, and such as categories (here these are species groups).

The images are labelled with multiple identifiers to make analysis and stratified sampling later simpler.

E.g,. **NO_2025_3007_GarnRuse_Lovund_LO3_RA1_L9_frame_9**, is collected in Norway, in 2025 on the 30.07 on the Garn and Ruse survey (gill- and fyke-net), the location name is Lovund, and is station number is LO3, and the BRUV-rig used is L9, and this is video number 9 from the left camera (L9), and is frame number 9. So if you are applying another type of video-survey or other naming logic please be mindful, though its possible to turn off this stratified sampling in the functions.

If you are annotating your own dataset, a .JSON file for the structure applied in this annotation process can be found in (`script/annotation_format`). For more in depth information about the sampling methods and areas, annotation choices and a discussion of methods can be found in Brøten [2026].

Chapter 2

Create a crab detector

The folder `create-a-crab-detector` contains sripts and datasets which makes its possible to go from raw annotations to a trained YOLO-model. The first script (`1-raw-to-yolo.ipnb`) takes raw CVAT annotations and turns them into a dataset ready for training an YOLO object detection model: RLE mask are converted into bounding boxes, and the data is split into five stratified folds for cross-validation.

If you're applying this dataset to other than the shipped example, keep in mind (but not limited to) - File paths: `CONFIG` needs to point at your own annotation JSON and image folder. - Stratified sampling for the k-folds, this assumes your images can be grouped by BRUV station and labeled crab-present/empty; if that doesn't apply to your data, this step may need adjusting or turning off (see Chunk 9).

By the end of the script, each image has a matching `.txt` file listing its bounding box annotations (or an empty file, if there's no crab in it), organized into five train/validation folds ready for object detection training.

2.1 From raw annotations to a YOLO model

1. Setup and loading data (Chunk 0 – 3)

These chunks handle setup: installing any missing Python packages, loading the required libraries, and locating your dataset. They should run without errors, if Chunk 3 can't find your annotation file or image folder, double check the paths.

If you're using your own dataset, update `CONFIG` in Chunk 2 with the correct paths to your annotation JSON and image folder.

2. Converting masks to bounding boxes (Chunk 4)

RLE masks are precise, but training directly on them takes more compute time and is really built for image segmentation, not simple object detection. Since the goal here is just to find and count crabs (not trace their exact outline), converting to bounding boxes is the better trade-off: they're faster to train on, and sufficient for detection.

RLE masks are also faster to draw in the first place, CVAT's Segment Anything–powered tool makes what's normally a tedious annotation process quicker.

This chunk converts each RLE mask into a tightly-fitted bounding box, and normalizes the box coordinates to the 0–1 range YOLO expects. Two filters are applied at the same time: annotations for any species other than Decapod: crab are discarded, and overlapping duplicate boxes (drawn twice for the same crab) are reduced to one.

3. Linking images to annotations (Chunk 5)

Links each annotation to its actual image file, matching by filename. If you're using your own dataset, make sure image filenames match the identifiers in your annotation JSON exactly, any mismatches get reported as "unmatched," usually caused by a typo, a renamed file, or an image missing from the folder.

Note: frames with no crab in them (negative examples) are deliberately kept in the dataset rather than discarded, training on empty frames as well as crab-present ones has been shown to improve performance [tror sorrenti er riktig kilde]. These get an empty `.txt` label file later, once the data is exported (Chunk 9).

4. Visual quality checks (Chunk 6 – 7)

These chunks are a visual quality check, not a processing step, and nothing here changes the data. Chunk 6 displays the first annotated image with its converted bounding box overlaid: every crab should have a red box tightly wrapping it, with no gaps or extra boxes.

Chunk 7 goes further, placing the original RLE mask and the converted bounding box for the same crab side by side, so you can visually confirm the conversion in Chunk 4 didn't distort or misplace anything before relying on it for training.

5. Location and station helpers (Chunk 8)

Filenames encode which location and BRUV station each image came from, but that information isn't stored as a separate field anywhere, it has to be pulled out of the filename itself. This chunk does that: `extract_location` reads out the site name (e.g. "Lovund"), and `extract_bruv_id` reads out the specific station ID (e.g. "LO3"). It also prints a quick summary: how many images come from

each location, and how many of those are crab-present versus empty, worth a glance to check whether your dataset is skewed toward a few sites, since that affects how balanced the folds can be.

If you're using your own dataset with a different filename convention, these two functions will need to be adjusted to match your naming pattern (see the Data section for the convention this project uses).

6. Splitting into stratified, grouped folds (Chunk 9)

This is the core of the export step: it splits the linked dataset into five folds for cross-validation, applying two rules at once.

Grouped: every image from the same BRUV station stays entirely in either training or validation within a fold, never split between the two. If the same station showed up in both, the model could effectively be tested on footage from a site it had already trained on, making it look like it's performing better than it actually would on a genuinely new site.

Stratified: within that constraint, each fold's train/validation split keeps a similar proportion of crab-present vs. empty frames as the full dataset, so no fold accidentally ends up training almost entirely on empty frames.

The function has a `stratify` option: `True` (the default) gives the behavior above; `False` skips the crab/empty balancing and keeps only the station grouping. Turning it off makes sense if your dataset is already fairly balanced, or if the crab/empty ratio varying between folds isn't a concern for you, station grouping still applies either way, so there's no risk of leakage regardless of this setting.

For each fold, this chunk writes the images and label files into `images/train`, `images/val`, `labels/train`, `labels/val`, along with a `dataset.yaml`.

YAML is a plain-text format for storing settings, written as simple, human-readable lines rather than code. YOLO's training tool reads `dataset.yaml` to know where to find the images and labels and what classes to expect. Each fold's file specifies: `path` (that fold's root folder), `train` and `val` (the image subfolders), `nc` (number of classes - 1, since this project only detects crabs), and `names` (mapping each class number to its name, e.g. `0: crab`).

7. Running the export (Chunk 10)

This chunk runs the function from Chunk 9 on your dataset. By default it creates five folds under `data/kfold/`, each structured as:

```
data/kfold/
  fold1/
      images/train/
      images/val/
```

```
    labels/train/
    labels/val/
    dataset.yaml
fold2/ ... fold5/          # same structure as fold1
master_dataset.yaml
fold_summary.csv
```

along with a `master_dataset.yaml`, a small reference file at the top level listing the class setup (`nc` and `names`, same as each fold's file) and a comment reminding you how to point training at a specific fold (e.g. `yolo train data=fold1/dataset.yaml ...`), and a `fold_summary.csv` with the actual per-fold stats mentioned above.

Once this runs cleanly, no station-overlap warnings, and a crab/empty ratio that looks reasonable in the fold summary, the data is ready for training in `train.ipynb`.

A caveat is that with this limited dataset of only ten stations, we are not expecting the folds to be perfectly balanced - and we can expect that the folds that look more unbalanced will perform worse than the others. Now you are ready to train!

2.2 Training the crab detector

This script (`train.ipynb`) takes the five-fold dataset produced by `raw_to_yolo.ipynb` and trains a YOLOv8 object detector on it, one model per fold, so performance can be evaluated across several independent train/validation splits rather than trusting a single one.

1. Setup (Chunk 1)

This chunk locates the repository (the same `find_repo_root()` approach used in `raw_to_yolo.ipynb`, so paths work the same whether you're running this on a laptop or a server), sets up where training output gets written, and checks that a GPU is available. Training a model like this on a CPU is impractically slow, so this check is worth paying attention to: if `CUDA available: False` prints here, stop and sort out the GPU setup before continuing rather than letting a full training run crawl for days.

2. Checking the folds (Chunk 2)

Before spending hours training, this chunk verifies every fold from `raw_to_yolo.ipynb` is actually complete: that each fold has a readable `dataset.yaml`, and that the number of images and label files line up in both the train and validation sets. This is a safety gate, not a formality, training on a broken or incomplete fold wastes real compute time, so it's worth letting

this fail loudly here rather than partway through a long run. Chunk 5 (the real training) refuses to start if this check doesn't pass.

3. Hyperparameters (Chunk 3)

This chunk sets out every training parameter in one place, so the whole run is reproducible from a single configuration. Rather than explaining every individual number, it's easier to think of them in three groups:

Model and image handling. `yolov8s.pt` is the pretrained starting point the model builds on rather than learning from scratch, you can freely try other sizes of YOLOv8 or other versions by changing them out here. Images are resized to 960 pixels and processed in batches of 16, larger images and batches use more GPU memory, but a larger image size in particular can help the model pick out small objects like crabs against a wide frame.

How the model learns. The optimizer, learning rate, and warmup settings control how aggressively the model adjusts itself while training, and how that pace changes over the course of the run (a "cosine" schedule, `cos_lr`, means the learning rate eases off smoothly toward the end rather than dropping abruptly). These values were carried over from what worked in the original thesis training runs, rather than re-tuned from scratch here.

Augmentation. During training, images are randomly rotated, shifted, scaled, and mirrored slightly before being shown to the model. The intent isn't to create unrealistic images, it's to stop the model from memorizing the exact footage it was trained on, so it generalizes better to new footage. A couple of these are deliberately constrained for this dataset: `flipud=0` means images are never flipped upside-down (a crab is never upside-down in real footage, so training on flipped versions would teach the model the wrong thing), and `single_cls=True` reflects that there's only one class to detect here (crab), so the model doesn't spend effort distinguishing between species that don't apply.

Two epoch counts are set here: `test_epochs` (5) for the quick sanity check in Chunk 4, and `epochs` (100) for the real run in Chunk 5, 100 was chosen because performance had already converged by that point in the original thesis training.

4. Test run (Chunk 4)

This runs a handful of epochs on one fold using the exact same settings as the full run, purely to confirm the pipeline works end-to-end, paths resolve, the GPU is used correctly, and training completes without errors before committing to a run that takes hours. The actual metrics from this run aren't meaningful and shouldn't be read as a preview of model quality: five epochs isn't enough for the model to properly learn, and results can look noisy or even get temporarily worse epoch to epoch. That's expected here, not a warning sign.

5. Full training (Chunk 5)

This trains one full model per fold, using the hyperparameters from Chunk 3. Each fold's best-performing checkpoint is saved automatically, along with a checkpoint every 25 epochs during training in case a run needs to be resumed. There's a short pause between folds to let the GPU cool down. The current server utilized is NVIDIA RTX A6000, which used around 3-4 hours for the entire training run, whilst training the full dataset used in the thesis took around 16–18 hours.

6. Collecting and interpreting results (Chunks 6 – 8)

Collection

Training five separate models means five separate sets of results, so these chunks pull them together: loading each fold's training log, plotting the key metrics (mAP, precision, recall) side by side across folds, and then computing a summary, both the final numbers from training and a fresh, explicit validation pass on each fold's saved model, as a double-check against the training log alone.

The reason to look across all folds rather than picking one is the same logic behind K-fold cross-validation in general: any single fold could happen to get an easy or a hard validation split by chance, so the story that matters is the overall pattern, not one number in isolation.

Interpretation

One limitation worth being upfront about: this dataset has only 10 BRUV stations, and because stations are kept fully separate between training and validation (to avoid the model being tested on footage it effectively already trained on – see the K-fold export step in `raw_to_yolo.ipynb`), each fold's validation set ends up being just 1-3 stations. That's a small, uneven sample, and fold-to-fold variation in these results should be read as a consequence of the dataset's size, not a flaw in the training method. One fold in particular had very few crab-positive images in its validation set purely by chance, and its numbers should be treated as unreliable rather than averaged in on equal footing with the others.

To evaluate object detection model performance, standard object detection metrics were used: true positives, false positives, false negatives, mean average precision (mAP50), precision, recall, and F1 (Table 2.1). After training, each model was validated on the validation dataset. The model predicts the class and location of objects in each frame with bounding boxes. To assign these metrics, predictions were ranked by confidence and matched to the manually annotated bounding boxes from the validation dataset, with each annotation matched to at most one prediction. A predicted box was counted as a true positive (TP) if it overlapped an annotation of the same class sufficiently, assessed using intersection-over-union (IoU); unmatched predictions

Table 2.1: Standard performance metrics for assessing object detection models. A detection is counted as correct (TP) if the predicted bounding box overlaps with an annotated box by at least 50% (IoU ≥ 0.5). Metrics range from 0 to 1, where 1 indicates perfect performance.

Metric	Description
True positive (TP)	Correct detections: predicted bounding box matches an annotated box.
False positive (FP)	Model predicts a bounding box where there is no object.
False negative (FN)	An object is present but the model produces no matching prediction.
Precision (0–1)	Ratio of correct positive predictions to all positive predictions, measures detec
Recall (0–1)	Ratio of correct positive predictions to all actual positives, measures the mode
F1 (0–1)	Harmonic mean of precision and recall, balances avoiding false detections with
Mean average precision (mAP50)	Average of precision scores across recall levels, summarized across all classes (

were counted as false positives (FP) and unmatched annotations as false negatives (FN) (a confusion matrix showing the distirbution of all these can be found in `test_runs/models/fold1_yolov8s_100epochs`. IoU is calculated as the area of overlap between the predicted and annotated box divided by the area of their union, giving a score between 0 and 1 [Biscarini and Nazzicari, 2026]. An IoU threshold of 0.5 was applied, meaning a prediction had to overlap at least 50% with the annotated box to be counted as correct, following standard object detection practice.

Aside for interpreting the performance metrics, its worth checking out the visual validation confirmations. In the folder `test_runs/models/fold1_yolov8s_100epochs`, or any other fold. Compare the `val_batch_labels.jpg` with `val_batch_pred.jpg`, this compares the actual annotated images and the predicted annotations from the model to see the confidence scores and what the model catches/misses. If the behavior seems wrong, there is a plethora of tweaks possible to improve model behavior that I wont touch on here but is easiliy available online or with the help of LLMs.

Chapter 3

Use a crab detector

3.1 Background and aims

The aim of this leg of the repository is to go from unseen data to assessment ready data, where unseen video material is the input and the output is the relative abundance indices MaxN and MeanN obtained with either the pretrained crab detector, or another model.

MaxN is the single image frame within a deployment with the highest count of individuals per deployment, wheras MeanN is the mean number of individual observed over a series of image frames [Whitmarsh et al., 2017]. MaxN is the most commonly applied metric fr estimating abundances in video-based surveys, and its conservative with its aim to reduce the likelihood of double counting individuals.

The pretrained crab detector developed in the thesis is included in this repository, in the folder `use-a-crab-detector/model/pretrained-crab-detector`.

A few caveats are worth mentioning. This model is not yet plug and play, as documented in the thesis, and in the literature more broadly, YOLO models in particular struggle to generalize: when the input material shifts away from the domain it was trained on, performance drops. Specifically, this model was trained to detect brown crab in BRUVs placed in nearshore coastal areas between 62°N and 68°N. When applied outside this scope, whether due to a different video collection method or a different environment, it's going to struggle (see the FjordFish test in Brøten [2026]).

If you'd still like to apply this detector given these caveats, or apply a new (and likely better) model, some manual work is still needed. To interpret the model's predictions, we need some understanding of how its generalizability. One important step is building an independent test set: annotate a small subset

of unseen data, let the model predict on it, then compare the model's predictions against the human-annotated ground truth.

3.2 Independent test set

Often described as the gold standard for evaluating a trained models performance, and is used after a model is fully trained. In this instance, its purpose is rather to evaluate the models "compatibility" with the data its going to predict on. All this is done in the script `get-ready-to-use-a-crab-detector`.

To start, you will need a fully annotated dataset, it does not need to be big, and is often around 10 - 25% of the available data. This data needs to be annotated, exported as COCO 1.0, and put in the folders, `use-a-crab-detector/data/test_predictions` prior to running the script.

Likewise to the data processing pipeline performed in `raw_to_yolo` the annotations are converted, and only the crab annotations are kept. The scrips logic for grouping is still based on the naming convention as previously mentioned, if the dataset uses another naming convention the code needs to be readjusted.

The script handles the comparison between the predicted bounding boxes to the individual annotated boxes frame by frame, the script summarizes crab counts per BRUV stations using MaxN and MeanN.

As the model predicts it assigns a confidence score to the prediction from $0 - 1$ based on how sure it is, the confidence threshold for predictions can be adjusted. High confidence thresholds keeps only the models most certain predictions (fewer false detections, but real crabs might be missed), wheras low confidence thresholds introduce retains more true predictions but introduces noise. Because there is no single correct threshold, the script evaluates several thresholds (none, 0.5 and 0.75 by default) so you can see the trade-off rather than guessing.

In the thesis, this comparison highlighted that neither a low or a high threshold was able to give an accurate MaxN prediction and highlighting the need for a more flexible solution for interpreting the detections, this is covered in the post-processing-pipeline.

Rather than comparing predicted boxes to individual annotated box.

3.2.1 Interpreting the performance metrics

Similar method to FF-setup GRAd-CAM analysis can be interesting to understand the results if the model struggled

3.3 Running the crab detector

Bibliography

Filippo Biscarini and Nelson Nazzicari. *Deep Learning for Life Sciences: with Python Notebooks for Examples and Exercises*, volume 1 of *Decoding Evolution*. Springer Nature Switzerland, Cham, 2026. ISBN 978-3-031-96851-8 978-3-031-96852-5. doi: 10.1007/978-3-031-96852-5. URL https://link.springer.com/10.1007/978-3-031-96852-5.

Gina Brøten. Crabs on camera: Improving the monitoring of the brown crab (*Cancer pagurus*) with machine learning and Baited Remote Underwater Video (BRUV). Master's thesis, University of Bergen, 2026. URL https://hdl.handle.net/11250/5537346.

Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO, January 2023. URL https://github.com/ultralytics/ultralytics.

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You Only Look Once: Unified, Real-Time Object Detection, May 2016. URL http://arxiv.org/abs/1506.02640. arXiv:1506.02640 [cs.CV].

Sasha K. Whitmarsh, Peter G. Fairweather, and Charlie Huveneers. What is Big BRUVver up to? Methods and uses of baited underwater video. *Reviews in Fish Biology and Fisheries*, 27(1):53–73, March 2017. ISSN 1573-5184. doi: 10.1007/s11160-016-9450-1. URL https://doi.org/10.1007/s11160-016-9450-1.